

Practical 5

- * Aim:- Create a PHP form to collect user details, validate inputs, and handle for submission securely.
- * Theory:- In web application development, collecting and storing user data securely is an important task. HTML forms are used to gather user information such as name, email, password, mobile number, & other details. In this practical, a php form was connected to collect user details, validate the inputs and securely store the information in a MySQL database created through the phpMyAdmin GUI.
- ① Data Collection using HTML forms:- An html form provides an interface between the user and the server. The form collects data entered by the user and sends it to the server for processing. For secure data transfer, the POST method is used instead of the Get method.
POST method sends data inside the HTTP request body, making it more secure for sensitive information.
- ② Database creation using phpMyAdmin GUI:-
In this practical, the database and table were created using the phpMyAdmin graphical interface instead of SQL queries.
- ③ Server-Side Validation and Sanitization:-
Client-side validation using HTML attributes

like required and type="email" improves user experience, but it is not completely secure because users can bypass it. Therefore, server-side validation in PHP is necessary.

* Validation:- It checks whether the entered data follows the required rules. Checking empty fields using empty(). Validating email format using:-

`filter_var($email, FILTER_VALIDATE_EMAIL)`

* Sanitization:- It removes harmful characters from user input before storing or displaying it. Functions like:- `htmlspecialchars()`

helps prevent script injection attacks by converting special characters into safe text.

(iv) Secure Database Insertion:- After validation, the data is inserted securely into the MySQL database using PHP and MySQLi prepared statements. Prepared statements are important because they protect the application from SQL Injection attacks. Instead of directly combining user input with SQL queries, placeholders are used and the data is safely inserted into them.

* Conclusion:- Thus, the PHP form was successfully created to collect and validate user details securely. The database created and the user detail was safely stored in MySQL using prepared statements. This practical helped in understanding securely handling form, validation and database connectivity.

Practical 6

- * Aim:- Implement Create, Read, Update and Delete (CRUD) operations on a user management system.
 - * Theory:- CRUD stands for Create, Read, Update and Delete, which are the basic database operations used in web applications. In this practical, the table was first created using the phpMyAdmin GUI, and then the database was connected to PHP using MySQLi. After establishing the connection, CRUD operations were performed on the user management system.
- ① Create Operation:- The create operation is used to add new user records into the database. A PHP form collects user details such as name, email and mobile number and stores them in the MySQL table.
 - ② Read Operation:- The read operation retrieves and displays user data from the database. PHP fetches records from MySQL and shows them in tabular format on the webpage.
 - ③ Update Operations:- The update operation modifies existing user records. Users can edit details through an update form, and the modified information is saved in the database.

④ Delete Operation :- The delete operation removes unwanted records from the database. PHP ~~st~~ sends a delete request using the user ID to permanently remove the selected records.

⑤ Database Management Using phpMyADMIN :- The database and table were created using the phpMy-Admin GUI by defining columns, data types, & constraints like Primary key and auto-Increment.

* Conclusion :- The CRUD operations were successfully implemented in the user management system using PHP and MySQL. The practical helped in understanding how to create, display, update and delete records securely while working with database connectivity and form handling.

Practical 7

- * Aim:- Create a file upload system in PHP where users can upload images and files.
- * Theory:- A file upload system allows users to upload images and documents from a webpage to the server. In this practical, PHP was used to create a secure file upload system where users can upload files such as images, PDFs and Documents.

① HTML file upload form:- An html form is used to select and upload files. The form uses:-

- ① method = "POST" for secure data transfer
- ② enctype = "multipart/form-data" to allow file uploading.

The file is selected using the `<input type="file">`.

② File handling in PHP:- PHP handles uploaded files using the `$_FILES` superglobal array. It stores information such as:-

- ① File name
- ② File type
- ③ File size
- ④ Temporary file location

Functions like `move_uploaded_file()` is used to move uploaded files from the temporary folder to a permanent server directory.

③ File validation :- Validation is important for security and proper file handling, PHP checks :-

Ⓐ File type (JPG, PNG, PDF, etc)

Ⓑ File size

Ⓒ Duplicate file names

This prevents uploading harmful or unsupported files.

④ Upload Directory :- A separate folder such as uploads/ is created to store uploaded files securely on the server.

* Conclusion :- Thus, the file upload system was created using PHP. Users were able to upload images and files securely through the web form. The practical helped in understanding file handling, validation, server storage and secure file uploading.

Education to Eternity